

**Character-Level Language Identification of Short Texts: A Transformer Baseline Versus
a Recurrent Encoder**

Arturo Ramos

Udacity

Master's Degree in AI Capstone — Project 4: Deep Learning Systems

Udacity Project Review Team

September 24, 2026

Character-Level Language Identification of Short Texts: A Transformer Baseline Versus a Recurrent Encoder

Report Overview

This project addresses language identification of short texts: given a snippet of 10 to 64 characters, predict whether it is Spanish, Portuguese, Italian, French, Romanian or Latin. The data come from the multilingual parallel Bible corpus of Christodoulopoulos and Steedman (2015), available at <https://github.com/christos-c/bible-corpus>. The baseline is a character-level Transformer encoder implemented in PyTorch, and the controlled experiment replaces only its encoder layer type with a bidirectional gated recurrent unit (GRU).

Dataset and Task Description

The corpus contains translations of the Bible in about 100 languages, encoded in XML and aligned by book, chapter and verse, and it is distributed under a CC0 license for research use (Christodoulopoulos & Steedman, 2015). This project uses six Romance-family languages, which together give 186,909 verse segments (between 31,100 and 31,292 per language); after removing 22 empty segments, 186,887 verses remain. Language identification is a well-studied task whose difficulty grows as texts get shorter and as the candidate languages get closer to each other (Jauhiainen et al., 2019), which is exactly the setting chosen here: six related languages and snippets as short as 10 characters, similar to the short posts a multilingual app must route to the right translation. The data were split by chapter into training (70 %), validation (15 %) and test (15 %) sets, so that no verse of a test chapter is seen during training in any language; this matters because the corpus is parallel and a verse-level split would leak the content of test verses through their translations. During training every verse is cut into a random window of 10 to 64 characters at each epoch; validation uses one fixed random-length window per verse, and the test set (26,769 verses) is evaluated at fixed lengths of 10, 20, 40 and 64 characters.

The initial inspection found several formatting artifacts tied to a single language that would let a model identify the language without learning it: the Portuguese file replaces the crase "à" with the Unicode replacement character in 2,402 verses (and contains no correct "à" at all), the backtick appears in 72.4 % of French and 20.5 % of Romanian verses, 25 Spanish verses contain an invisible soft hyphen, and 96.0 % of Latin verses start in lowercase against at most 15.7 % in the other languages. Normalization restored the crase, replaced backticks with apostrophes, removed soft hyphens and lowercased all text, so that only real letters and words remain as evidence.

Model Architecture and Design Decisions

Characters rather than words were chosen as the input unit because short snippets often cut words in half and contain few complete words, and because character sequences such as endings, digraphs and diacritics are among the strongest cues for distinguishing related languages (Jauhiainen et al., 2019). The vocabulary contains the 83 characters seen at least five times in the training set, and each snippet becomes a padded sequence of 64 character ids. Both models share the same input layer, a 128-dimensional character embedding plus a learned position embedding, the same masked mean pooling over the real characters, and the same linear output layer to six classes trained with cross-entropy. The baseline encoder is a Transformer (Vaswani et al., 2017) with two layers, four attention heads, a feed-forward width of 256 and dropout of 0.1, for a total of 284,550 trainable parameters. Self-attention lets every character attend to every other character of the snippet, with padding positions masked. Dropout (Srivastava et al., 2014) is used in the embeddings and inside the encoder to reduce overfitting.

Both models were trained with the same setup: the AdamW optimizer, which decouples weight decay from the gradient update (Loshchilov & Hutter, 2019), a learning rate of 0.001, batches of 256 snippets and 12 epochs. After each epoch the model was evaluated on the validation set and the weights with the best validation accuracy were kept, a form of early

stopping that protects the final model from late overfitting (Goodfellow et al., 2016). Every random generator was seeded, PyTorch's deterministic algorithms were enabled and the non-deterministic fused attention kernels were disabled so that results can be reproduced.

Experimental Comparison

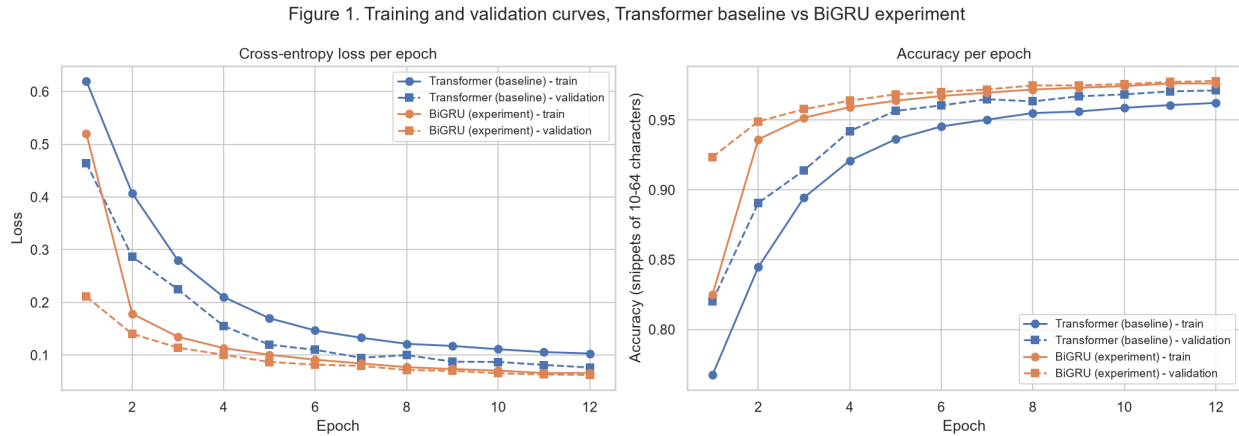
The experiment changes exactly one major aspect of the baseline: the encoder layer type. The two Transformer encoder layers are replaced by one bidirectional GRU layer with 64 units per direction, whose 128-dimensional output matches the width of the Transformer, so the pooling and output layers are unchanged. The GRU is a gated recurrent unit that reads a sequence one element at a time and keeps a hidden state that summarizes what it has seen (Cho et al., 2014); running it in both directions gives every position context from both sides. Everything else stays the same: data, split, vocabulary, snippet sampling, embeddings, pooling, loss, optimizer, learning rate, batch size, number of epochs, seed and checkpoint selection. The comparison is meaningful because it isolates the effect of the architectural inductive bias. Identifying the language of a short snippet depends mostly on local character patterns, which a recurrent layer processes in order by construction, whereas a Transformer must learn locality from data through attention and position embeddings. The experiment therefore tests whether that built-in bias helps a small model on this task, and at what cost in parameters and training time.

Results and Interpretation

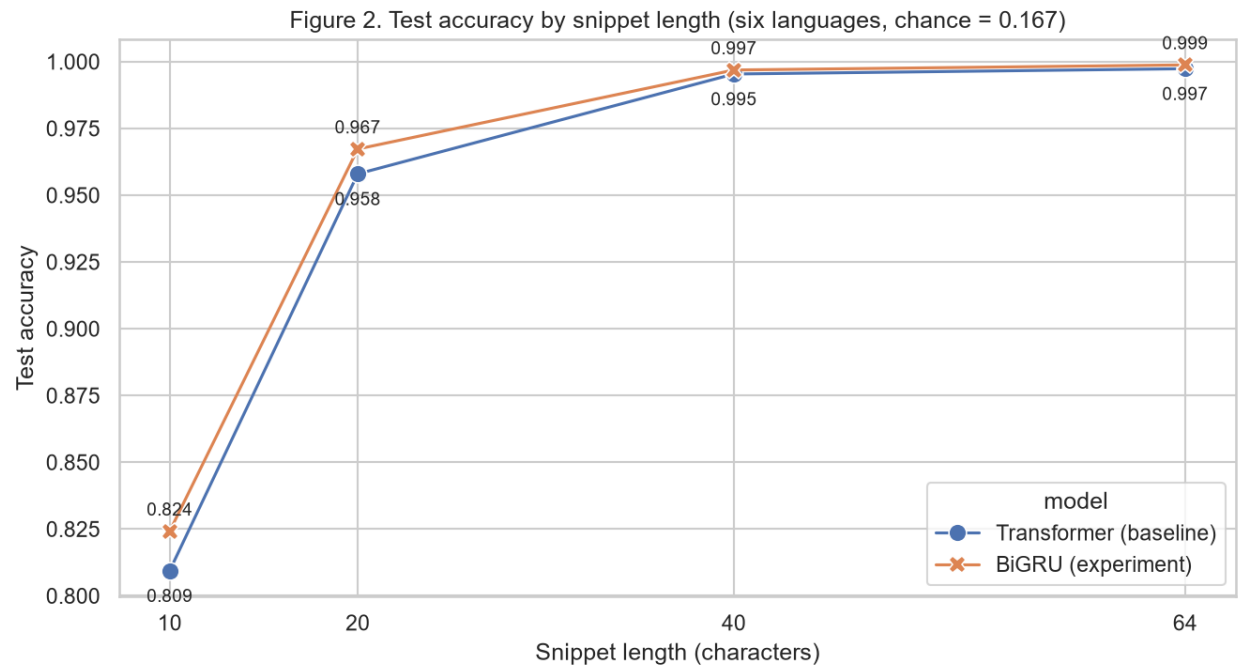
The BiGRU experiment outperformed the Transformer baseline at every snippet length (Figure 2). With 10 characters, test accuracy was 0.8242 for the BiGRU against 0.8094 for the Transformer; with 20 characters, 0.9673 against 0.9580; with 40 characters, 0.9969 against 0.9954; and with 64 characters, 0.9988 against 0.9973. Macro-F1 followed accuracy closely because the six languages are almost perfectly balanced. The BiGRU achieved this with a third of the parameters (94,086 against 284,550) and less training time (140 against 252 seconds on the GPU in the submitted run, 55 % of the baseline's time; wall-clock times vary between

runs), and it also reached a higher best validation accuracy (0.9780 against 0.9712). Because both models were scored on the same snippets, the differences were tested with McNemar's exact test on the paired predictions, which considers only the snippets on which the two models disagree (McNemar, 1947). The BiGRU was significantly better at every length ($p < 0.001$; at 10 characters it was the only correct model on 1,942 snippets against 1,547 for the Transformer), but at 40 and 64 characters both models are near the ceiling and the absolute gain is only 0.15 and 0.14 percentage points. These tests compare one trained model of each kind, so they do not capture variability across random seeds. The advantage is largest exactly where the task is hardest, on the shortest snippets, which supports the hypothesis that a bias toward local character order is useful when there is little text to attend over.

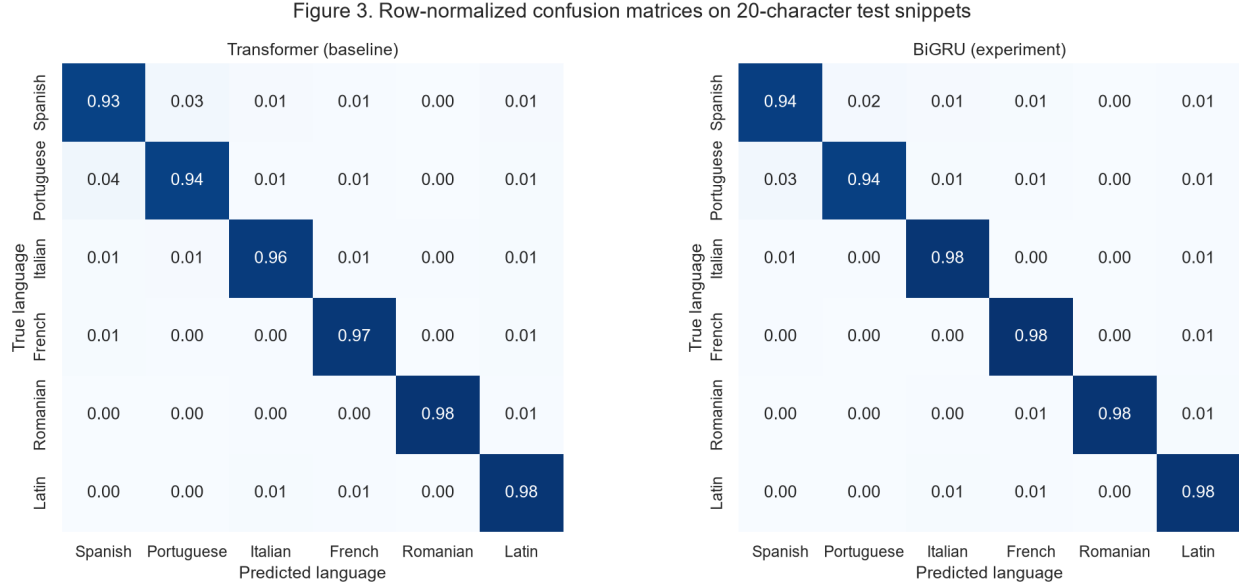
The training curves (Figure 1) show a concrete aspect of model behavior: neither model overfits within 12 epochs. Validation accuracy is slightly higher than training accuracy for both models (final gaps of -0.0089 for the Transformer and -0.0018 for the BiGRU), because training accuracy is measured with dropout active on random crops, and the best validation score was reached at the last epoch. The Transformer starts from a higher loss and improves more slowly, and both curves are still rising slowly at the end, so the models are, if anything, slightly under-trained. The dominant factor is snippet length: both models are nearly perfect with 40 or more characters, but with 10 characters about one snippet in six is misclassified. The confusion matrices on 20-character snippets (Figure 3) show that errors concentrate between Spanish and Portuguese, the closest pair (155 Portuguese snippets predicted as Spanish and 103 Spanish snippets predicted as Portuguese by the BiGRU), and toward Italian, while Romanian, with its distinctive letters, has the highest per-language F1 (0.987) and Spanish the lowest (0.948). Several of the misclassified 10-character snippets are genuinely ambiguous rather than model failures: *servidores* is the same word in Spanish and Portuguese, and snippets that cut a word in half lose the ending that identifies the language.

Figure 1*Training and Validation Loss and Accuracy per Epoch for Both Configurations*

Note. Solid lines are training, dashed lines validation; validation uses one fixed snippet of 10–64 characters per verse.

Figure 2*Test Accuracy by Snippet Length for the Transformer Baseline and the BiGRU Experiment*

Note. 26,769 test snippets per length from held-out chapters; chance level is 0.167.

Figure 3*Row-Normalized Confusion Matrices on 20-Character Test Snippets*

Note. Each row sums to 1; off-diagonal cells show which languages are confused.

Limitations and Risks

The most important limitation is the domain. All training and test text comes from Bible translations, which use a formal, religious and in some cases archaic register (the Romanian edition uses the spelling in force before 1993, with $\hat{i}$ where current Romanian writes $\hat{a}$, and Latin is a classical language), so the reported accuracy is an estimate for this kind of text, not for everyday social-media posts with slang, spelling mistakes, emoji or code-switching between languages. The chapter-level split prevents content leakage, but the test chapters still come from the same six translations, so the evaluation does not measure how the models behave on other translations or authors. The label is also imperfect for very short inputs: a 10-character snippet can be a valid string in several of these languages, which caps the achievable accuracy and makes part of the error unavoidable. The comparison is limited to one training budget of 12 epochs and one random seed per configuration; because both models were still improving, a longer schedule might narrow or change the gap between them.

Changing the layer type also changed the depth (two Transformer layers against one GRU layer) and the capacity (284,550 against 94,086 parameters), so part of the GRU's advantage could come from a smaller model converging faster within the same budget rather than from the recurrent inductive bias alone; the experiment changes one aspect, the architecture, but it cannot separate these two explanations. Finally, the repair of the Portuguese encoding error assumes that every replacement character stands for the crase, which is true for almost all occurrences but introduces a few wrong letters in words such as não.

Ethical and Responsible Use

A language identifier decides which people's text is processed correctly, so its errors are not neutral. In this model the errors are unevenly distributed: speakers of Spanish and Portuguese are misclassified more often than speakers of Romanian, and a Portuguese-speaking user whose short post is labeled as Spanish would receive a wrong automatic translation or be routed to the wrong moderation queue. Research on language identification has shown that such systems systematically perform worse on dialects and on the language of underrepresented groups, and that training data must include that variation to avoid excluding those users (Jurgens et al., 2017). The present model was trained on a single formal variety per language, taken from one Bible translation each, and has never seen informal or regional writing, so deploying it in a Latin American community app without further data would risk exactly this bias. There is also a risk of misuse in overconfident automation: a snippet of 10 characters carries little evidence, so a responsible system should not act on the prediction alone. A reasonable safeguard is to act automatically only on longer inputs or high-confidence predictions, to ask the user or keep the original text when the model is uncertain, and to report per-language error rates rather than a single overall accuracy. The data themselves raise a copyright consideration: some translations in the corpus are still protected, so the files are downloaded from the research corpus for this analysis and are not redistributed.

Future Improvements

Four next steps would make the system more reliable. First, evaluate and fine-tune on real short posts written by users, including informal and regional varieties of Spanish and Portuguese, to measure and reduce the domain gap. Second, train longer with a learning-rate schedule and several random seeds per configuration, to confirm that the advantage of the recurrent encoder is stable and not an effect of the 12-epoch budget. Third, add a calibrated confidence score and an "uncertain" output for very short inputs, so that downstream features such as automatic translation are triggered only when the prediction is reliable. Fourth, compare against a character n-gram baseline, the traditional method for language identification (Jauhiainen et al., 2019), to quantify how much the neural models add for this set of languages.

References

- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734). Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1179>
- Christodoulopoulos, C., & Steedman, M. (2015). A massively parallel corpus: The Bible in 100 languages. *Language Resources and Evaluation*, 49(2), 375–395. <https://doi.org/10.1007/s10579-014-9287-y>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org>
- Jauhiainen, T., Lui, M., Zampieri, M., Baldwin, T., & Lindén, K. (2019). Automatic language identification in texts: A survey. *Journal of Artificial Intelligence Research*, 65, 675–782. <https://doi.org/10.1613/jair.1.11675>
- Jurgens, D., Tsvetkov, Y., & Jurafsky, D. (2017). Incorporating dialectal variability for socially equitable language identification. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 51–57). Association for Computational Linguistics. <https://doi.org/10.18653/v1/P17-2009>
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Bkg6RiCqY7>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958. <https://jmlr.org/papers/v15/srivastava14a.html>

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.

<https://doi.org/10.1007/BF02295996>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998–6008). Curran Associates. https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html